



Cairo University Racing Team

Formula Student / FSAE®

Generative AI Team

(Technical Task)

Season 26-27

1. Project Description

This project requires you to build a small assistant that can answer questions about a CURT inventory. The assistant should let a team member ask things like "how many brake pads do we have left" or "where is the ECU stored" and get a useful answer back.

The inventory itself will live in a database, and the assistant will be wrapped in a simple Streamlit interface.

There are two stages to this task:

- **Phase 1:** A rule-based assistant with no external AI model.
- **Phase 2:** The same assistant, now powered by a real LLM API with function calling and a backend service, able to understand natural-language questions instead of fixed keywords.

2. Tasks to Complete the Project

1. Set up the inventory database.
2. Build the Phase 1 rule-based assistant.
3. Build the Phase 2 LLM-powered backend.
4. Build the frontend with Streamlit.
5. Handle edge cases.
6. Write a short reflection.

Please tackle as many tasks as you feel comfortable completing; do as much as you are able to.

3. Detailed Description of Each Task

Task 1: Set Up the Inventory Database

Objective: Represent CURT's parts inventory in a simple SQL database.

Steps:

- Use SQLite as the default database. You may use MySQL/PostgreSQL if you prefer them.
- Create a database (curt_inventory) and a parts table.

- Include at minimum: **id, name, quantity, category, location**.
- Seed the table with at least 10 realistic parts.

Example SQL to create the table:

```
CREATE DATABASE curt_inventory;  
USE curt_inventory;  
  
CREATE TABLE parts (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(255),  
  quantity INT,  
  category VARCHAR(100),  
  location VARCHAR(100)  
);
```

- Write a small data-access layer that your assistant will call instead of writing raw SQL — e.g. `get_part(name)`, `get_by_category(category)`, `update_quantity(name, delta)`.
- Both Phase 1 and Phase 2 should read from this same database, not a separate copy.

Task 2: Build the Phase 1 Assistant

Objective: Build an assistant that answers a fixed set of question types using only your own logic (if/else, keyword matching, or a simple parser) — no AI model involved. It should query the real database from Task 1.

Steps:

- Accept a typed question from the user.
- Support at least these query types:
 - "How many [item] do we have?"
 - "Where is the [item]?"
 - "List all items in [category]."

Security requirements:

- API keys must not be hardcoded in the source code.
- Use environment variables (e.g. .env) for secrets which is **not** to be committed to github repository.
- The LLM must not have direct access to the database.
- Database access must happen through controlled backend functions/tools.

Return a clear, correctly formatted answer for each, pulled live from the database.

Task 3: Build the Phase 2 Assistant (LLM-Powered Backend)

Objective: Build a real backend service that lets an LLM answer natural-language inventory questions using function calling, with multi-turn conversation memory.

Steps:

- Build an API server: Use Flask or FastAPI with at least these endpoints:
 - **POST /chat** → accepts a user message and a session/conversation id, returns the assistant's reply.
 - **GET /inventory** → returns the current inventory as JSON.
- Use function calling: Give the model tools it can call, and have your backend execute them against the real database from Task 1:
 - **check_stock(item_name)** → queries the database for quantity + location.
 - **list_by_category(category)** → queries the database for all items in that category.
 - **flag_shortage(item_name)** → logs a "low stock" flag (print/log is enough, no real alerting needed).
- The model should decide when to call which function based on the user's question.
- Maintain conversation memory per session. An in-memory dict keyed by session id, holding the message list, is enough, no persistence is required.

- The assistant should support follow-up questions that depend on previous context. For example:

User: How many brake pads do we have?

Assistant: We have 12 brake pads.

User: Where are they stored?

Assistant: They are stored in the Mechanical Workshop.

- You may use any LLM API that supports function/tool calling. Google Gemini, OpenAI, or another suitable provider are acceptable. **Google Gemini** is recommended as it offers a free tier option for this task.

Task 4: Build the Frontend with Streamlit

Objective: Give the assistant a simple, usable interface instead of a terminal.

Steps:

- Build a single-page Streamlit app with:
 - A chat-style input box where the user types a question.
 - A scrolling display of the conversation (question + answer pairs).
- Add a sidebar or expandable panel that shows the current inventory as a table, pulled live from the database — this is just for demo/debugging convenience, not a new feature.
- If you have built both phases, add a toggle or dropdown so the reviewer can switch between "Phase 1 (rule-based)" and "Phase 2 (LLM)" without restarting the app.
- If you only built Phase 1, the Streamlit app should still call your real Phase 1 logic against the real database.

Task 5: Handle Edge Cases

Your assistant should behave sensibly when:

- The requested item does not exist in the database.
- The user misspells or partially types an item name.

- A question is ambiguous or missing information (e.g., "how many do we have?" with no item named).

How you choose to handle each of these is intentionally left up to you, there is no single correct behavior expected. Document the choice you made and why.

Task 6: Reflection

Write a short reflection covering:

- The edge-case decisions you made and your reasoning.
- What you would improve or add with more time.

4. General Instructions & Submission

1. Submit your complete source code via a public GitHub repository dedicated to the CURT Inventory Assistant.
2. A comprehensive **README.md** is required and must detail: project architecture, tech stack (SQLite, Flask/FastAPI, Streamlit, LLM provider), local environment setup, SQLite seeding/migration instructions, API endpoint documentation (/chat, /inventory), tool-calling definitions, and any edge-case handling decisions.
3. **Bonus Task** – Railway Deployment: You may deploy the application using Railway as an optional bonus. If you choose not to deploy the project, this will not affect the core submission; however, the README.md must clearly explain how to install, configure, and run the project locally.
4. Record a 2-5 minute video demonstration showing Phase 1 & Phase 2 query execution in Streamlit, live database inspection, tool-calling execution flow, and a code walkthrough; share via a public Google Drive link.
5. Send an email with the subject “Generative AI Task Submission - [Your Name]” including your GitHub repository link, video link, and a sample .env.example file (never commit actual API keys).
6. Using external documentation, API references, and open-source libraries for Flask/FastAPI, Streamlit, SQLite, and LLM SDKs is permitted and encouraged.
7. You may choose Flask or FastAPI for the backend API, and any LLM provider supporting function calling (e.g., Google Gemini API, OpenAI API).

8. All function calling logic, data access abstraction layers, and Streamlit components must reflect your own code design and understanding; you must be prepared to explain your system design and tool-calling implementation during code review.
9. The project deadline is **5 days** from assignment receipt.
10. For any questions during the task, contact us via **sda.curt@gmail.com**